TERADATA TO DATABRICKS (WITHOUT REGRETTING IT LATER)

A dose of realirt

FEB 05, 2026

22 4 3 Share

Moving from Teradata to Databricks (Without Regretting It Later)

By Jeff Wilts

At some point, someone in your organization will stand up in a meeting and say something like:

"We're moving off Teradata. Databricks is modern. It's cheaper. It's faster. And we can finally stop worrying about all that old-school data warehouse discipline."

This usually sounds very confident. It is also usually partially wrong.

What follows is not a story about technology failure. Databricks is an impressive platform. Teradata is a mature one. Both do exactly what they are designed to do. The failures happen because people assume they are philosophically the same thing. They are not.

Teradata is a full-featured enterprise data warehouse. Databricks is a unified data platform that *can* behave like a data warehouse — **if** you deliberately make it do so. That "if" is where most migrations can go off the rails.

Databases encode assumptions. When you change platforms without understanding those assumptions, you don't get a clean modernization. You get what I like to call a *Lakehouse Liability*: something shiny, expensive, and possibly oddly untrustworthy.

Let’s talk about why.

Built-In Discipline vs. Earned Discipline

Teradata enforces a lot of discipline for you, whether you like it or not.

Declare a primary key? Uniqueness is enforced.
Define a SET table? Duplicates are rejected.
Create triggers? They fire exactly when rows change.
Use views? You can insert, update, and layer them in surprisingly powerful ways.

Databricks does none of this by default.

Primary keys are informational.
Uniqueness is a suggestion.
Triggers exist, but not in the way Teradata people expect.
Views are read-only abstractions.

This is not a flaw. It is a design choice.

Databricks assumes you will manage correctness through pipelines, patterns, and governance. Teradata assumes correctness is the database’s job. If you don’t notice this difference early, you will notice it later — usually during a reconciliation meeting where nobody is smiling. This is earned discipline. Not optional discipline.

Example: Primary Keys

Teradata

CREATE TABLE customer (

> customer_id INTEGER PRIMARY KEY,
>
> name VARCHAR(100));

Duplicate insert:

> INSERT INTO customer VALUES (1, ‘Alice’);

> INSERT INTO customer VALUES (1, ‘Bob’);

Result: **Error**. Constraint enforced.

Databricks

CREATE TABLE customer (

> customer_id INT,
>
> name STRING,
>
> CONSTRAINT pk_customer PRIMARY KEY (customer_id));

Same duplicate insert:

> INSERT INTO customer VALUES (1, ‘Alice’);
>
> INSERT INTO customer VALUES (1, ‘Bob’);

Result: **Both rows succeed**.

The constraint is informational. It helps tools. It does not protect your data.

So you enforce uniqueness in pipelines:

MERGE INTO customer t

USING staging_customer s

ON t.customer_id = s.customer_id

WHEN MATCHED THEN UPDATE SET *

WHEN NOT MATCHED THEN INSERT *

Discipline moves from engine → pipeline.

If you don’t redesign for that, duplicates will accumulate quietly until someone runs a reconciliation.

Example: Duplicate Rejection

Teradata SET tables reject duplicates automatically.

```
CREATE SET TABLE orders (

order_id INTEGER,

amount DECIMAL(10,2) );
```

Databricks equivalent behavior must be simulated:

```
CREATE OR REPLACE TABLE orders_clean AS

SELECT DISTINCT *

FROM staging_orders;
```

Or enforced in streaming:

```
df.dropDuplicates(["order_id"])
```

Collation: The Quiet Way to Break Trust

Collation is one of those topics everyone skips because it sounds boring.

Until joins start dropping rows.

Collation defines how text is compared and sorted: case sensitivity, accent handling, language rules. Teradata gives you extensive control over this at the database, user, and column level. Databricks largely relies on UTF-8 binary comparisons: fast, simple, and unforgiving.

Teradata often behaves case-insensitively. Databricks often does not.

If you don't explicitly decide how text comparisons should behave, you will still get a decision — just not one you made consciously. The result is dashboards that looks like it "randomly" change, distinct counts that drift, and analysts who start adding workarounds downstream.

Once trust is lost, performance improvements stop mattering.

Example: Case Sensitivity

Teradata

```
SELECT *

FROM customers

WHERE name = 'alice';
```

May match: Alice, ALICE, alice

Databricks default

```
SELECT *

FROM customers

WHERE name = 'alice';
```

Matches only exact case.

So this join:

```
SELECT *

FROM a

JOIN b

ON a.name = b.name;
```

can silently lose rows after migration.

Mitigation pattern:

```
SELECT *

FROM a
```

JOIN b

ON LOWER(a.name) = LOWER(b.name);

But now you've introduced:

· CPU overhead

· semantic drift

· inconsistent behavior across teams

The correct solution is not sprinkling LOWER() everywhere. The correct solution is a **collation strategy** and documented comparison rules.

Data Types: "It Loaded" Is Not the Same as "It's Right"

Teradata's type system is strict, expressive, and full of edge cases people forgot they were relying on.

BYTEINT.
INTERVAL.
High-precision DECIMAL.
TIMESTAMP WITH TIME ZONE.

Databricks supports many of these concepts, but not always in the same way, and not always as first-class citizens. Types get promoted. Precision limits matter. STRING becomes the universal escape hatch.

The classic failure looks like this:

You migrate a fact table.
The data loads cleanly.
The rollups run.
The totals are off by a very small amount.

That "very small amount" will cost you in credibility.

Floating-point math behaves differently. Implicit casts behave differently. Rounding behaves differently. None of this is obvious until someone compares old numbers to new ones and asks the most dangerous question in analytics:

“Which one is correct?”

If you don’t want that conversation, build a type-mapping matrix *before* you migrate and test the ugly cases deliberately.

Example: DECIMAL Precision Drift

Teradata fact table:

```
amount DECIMAL(38,10)
```

Spark aggregation:

```
SELECT SUM(amount)

FROM fact_sales;
```

If implicit casting occurs:

- precision loss
- floating behavior differences
- rounding changes

A 0.01% drift becomes a credibility problem.

Spark Mitigation:

```
SELECT CAST(SUM(amount) AS DECIMAL(38,10))

FROM fact_sales;
```

And validation:

```
SELECT old_sum, new_sum, old_sum - new_sum AS delta

FROM reconciliation_table;
```

You don't test averages.

You test pathological rows.

Golden rows:

- max precision
- negative values
- null combinations
- extreme timestamps

Example INTERVAL

Teradata:

INTERVAL '01:30:00' HOUR TO SECOND

Databricks workaround:

duration_seconds BIGINT

Or ISO string:

duration STRING -- "PT1H30M"

The Data Model: The Discipline You Have to Rebuild on Purpose

Teradata explicitly forces you to think in terms of a data model. Not because it's philosophical. Because if you don't, things break. Keys matter, relationships matter, table design has consequences.

When teams move to Databricks, one of the first invisible casualties can be the data model — not because anyone deletes it, but because the platform no longer insists on it. The engine will happily accept loosely related tables, duplicate identifiers, drifting semantics, and "temporary" structures that quietly become permanent.

Nothing may explode immediately. That's what makes it dangerous. A data model is not just a diagram someone made once for documentation. It is the explicit statement of what the business believes is true: What is a customer? What makes an order unique? Can an account exist without an owner? What does "active" mean?

Teradata encodes these beliefs structurally. Databricks assumes you will encode them socially — through conventions, pipelines, and shared understanding.

That assumption only works if you are deliberate.

Example: Identity Drift

Imagine a customer table without enforced uniqueness:

customer_id | email
1 | alice@example.com
1 | alice@workmail.com

Both rows load. Both rows propagate. Both rows show up in analytics.

Now every downstream metric involving "customer count" becomes an opinion instead of a fact. The data model is what prevents identity from becoming negotiable.

In a lakehouse environment, the model becomes a contract enforced by process instead of database constraints:

· MERGE rules define identity

· promotion pipelines validate uniqueness

· quality checks reject semantic violations

· published tables represent certified truth

If you don't formalize that discipline, the platform will not do it for you.

Bronze/Silver/Gold architectures are not just performance layers. They are modeling layers:

Bronze = raw events
Silver = conformed entities
Gold = business meaning

The data model is what turns data into entities and entities into decisions.

Without it, you don't have a lakehouse. You have a very fast junk drawer.

Ontology: When the Problem Isn't Structure — It's Meaning

A data model answers: How is the data structured? An ontology answers a harder question: What does the data mean? Two teams can share the same schema and still disagree about reality.

That is not a technical problem. It is a semantic one. An ontology is a shared vocabulary that defines concepts, relationships, and intent. It sits above the schema and prevents meaning from drifting as the system grows.

In small systems, ontology lives in people's heads. In large systems, that stops scaling.

Example: The Word "Customer"

> · Marketing customer: someone who signed up
>
> · Billing customer: someone who paid
>
> · Support customer: someone with an open ticket
>
> · Data warehouse customer:
> whoever the ETL decided last quarter

Same table name. Different ontology. The schema didn't fail.
The shared meaning did.

Lakehouse platforms amplify this problem because they remove central bottlenecks. More teams can publish data faster. That's good for velocity. It's terrible for semantic consistency if you don't define a common language.

An ontology gives you:

· shared definitions

· stable business concepts

· explicit relationships

· machine-readable meaning

This matters for more than humans. Downstream systems increasingly rely on semantic clarity:

· BI layers

· metrics stores

· data catalogs

· AI models

· governance tooling

If your ontology is implicit, every tool invents its own. That's how organizations end up arguing over dashboards instead of decisions.

Practical Ontology Pattern

· Define canonical entities: Customer, Order, Product, Subscription

· Define their relationships: Customer places Order, Order contains Product Customer owns Subscription,

· Align schemas to those concepts instead of letting each pipeline improvise terminology.

Unfortunately Databricks Unity Catalog does not have native capabilities for capture and managing an ontology layer. It robustly manages technical frameworks for data governance (access, control, auditing, lineage, physical schema) but you will need a different solution for business context.

You are not adding bureaucracy. You are preventing semantic entropy. The bigger and more flexible the platform becomes, the more important ontology becomes. Not because technology demands it — but because humans do.

Databricks removes structural guardrails. Ontology restores conceptual ones. Without it, the system still runs. It just slowly stops agreeing with itself.

Transactions: ACID Is Not a Yes/No Question

Teradata users are used to wrapping a sequence of statements in a transaction and expecting all-or-nothing behavior.

Databricks gives you ACID guarantees — but typically at the level of a single operation.

That distinction matters when your pipeline does something like:

- update a dimension
- insert facts
- write audit rows
- publish a completion flag

If step three fails, you may have just published a half-truth.

The solution is not to complain that Databricks is “missing” something. The solution is to design for atomicity explicitly: MERGE patterns, publish-from-staging approaches, and idempotent pipelines that can be safely re-run.

Safer Pattern: Staging Publish in Databricks

```
CREATE TABLE fact_staging AS

SELECT ...

Validate:

SELECT COUNT(*) FROM fact_staging;
```

Publish atomically:

REPLACE TABLE fact_production AS

SELECT * FROM fact_staging;

Or Delta version swap.

Time Zones: Decide Early or Pay Forever

Time zones are exhausting. Everyone knows this. That's why teams postpone decisions about them.

Then daylight savings hits.

Teradata supports time-zone-aware timestamps natively. Spark historically treats timestamps as instants interpreted through session settings unless you deliberately use non-time-zone types.

You must decide what your timestamps *mean*.

Are they instants in time? Store UTC.
Are they wall-clock business times? Store local time explicitly.
Are they mixed? Store both the instant and the original offset.

Assuming "timestamp is timestamp" guarantees pain later. This is not a technical nuance. It is a semantic contract.

Best Practice Pattern

Store event instant:

event_ts_utc TIMESTAMP

Store original zone:

event_tz STRING

Display conversion:

SELECT from_utc_timestamp(event_ts_utc, event_tz)

What Happens If You Don't Decide

- DST (Daylight Saving Time) shifts
- audit drift
- inconsistent reporting
- analysts invent fixes

SQL Differences: The Death of a Thousand Paper Cuts

Most Teradata SQL ports cleanly. The rest may make you question your life choices.

Volatile tables.
Macros.
Triggers.
Update-with-join patterns.
Stored procedures.

Each one is manageable. Collectively, they demand a systematic approach. Decide early how procedural logic will be handled: Databricks SQL scripting, notebooks, dbt, or something else. Inconsistency here is how maintenance costs quietly explode.

Example: Update Join Rewrite

Teradata:

UPDATE target t

FROM source s

SET t.value = s.value

WHERE t.id = s.id;

Databricks:

```
MERGE INTO target t

USING source s

ON t.id = s.id

WHEN MATCHED THEN UPDATE SET value = s.value;
```

Example: Volatile Tables

Teradata:

```
CREATE VOLATILE TABLE temp AS (...);
```

Databricks:

```
CREATE OR REPLACE TEMP VIEW temp AS (...);
```

Or:

```
df.createOrReplaceTempView("temp")
```

Don't Just Rebuild Teradata on Databricks

One of the most common mistakes is recreating the Teradata warehouse exactly as it was, just on new infrastructure.

Same schemas.
Same layering.
Same mental model.

You *can* do this. You just won't get much benefit from it. Databricks rewards discipline differently: layered Bronze/Silver/Gold models, explicit publish patterns, governance through Unity Catalog, and data quality checks as part of promotion — not as an afterthought.

The lakehouse does not eliminate the need for discipline. It removes the guardrails that forced it.

Final Thought

Teradata spent decades teaching organizations hard lessons:

· Typing matters.

· Semantics matter.

· Governance matters.

· “It runs” is not the same as “it’s right.”

Databricks does not invalidate those lessons. It simply gives you a much larger platform on which to relearn them — faster, louder, and at cloud scale — if you are not careful.

The devil is not in the details of the technology. The devil is in the assumptions and decisions you didn’t realize you were making.

Jeff Wilts is a Chief Architect and senior technology executive with 25+ years of hands-on experience designing, modernizing, and scaling enterprise data, AI, and platform architectures across FinTech, IoT, retail, and large digital marketplaces.

Currently a VP and Chief Architect in the Data & AI Practice at Ness, he is engaged in one of the world’s largest and most complex Teradata to Databricks modernization programs. Jeff has deep expertise in distributed systems, cloud-native architectures, data engineering, M&A carve out integrations, and regulated data environments, and has built and led global teams delivering high-availability, mission-critical systems. His work bridges architecture strategy and execution, with a strong emphasis on operationalizing data, AI/ML, and automation to drive reliability, performance, and measurable business outcomes.

Subscribe to William’s Substack

By William Inmon · Launched 5 months ago

My personal Substack

slysik@gmail.com Subscribe

By subscribing, you agree Substack's Terms of Use, and acknowledge its Information Collection Notice and Privacy Policy.

22 Likes · 3 Restacks

Discussion about this post

Comments Restacks

Write a comment...

Donald Parish Feb 5

Good article. I miss “real” databases.

LIKE (2) REPLY SHARE

Martyn Jones Feb 6

This is purely about end-user (IT department) technology.
https://goodstrat.com/2026/02/06/understanding-the-data-warehouse-dilemma-2026-02-07/

LIKE REPLY SHARE

2 more comments...

© 2026 William Inmon · Privacy · Terms · Collection notice
Substack is the home for great culture